ENGENHARIA DE SOFTWARE / DEVSECOPS AND CLOUD COMPUTING

FIAP - CP05
Pipeline DevSecOps Completo

Gitleaks · Semgrep · Trivy

Checkpoint 05 | 13 de outubro de 2026

Equipe — cinco integrantes
Enzo Vazquez Sartorelli — RM94818
Guilherme Brazioli — RM98237
Pedro Oliveira Valotto — RM551445
Nicolas Nogueira Boni — RM551965
Kaue Pastori Teixeira — RM98501

Repositório: github.com/efsartorelli/CP05-DEVOPS

01 / VISÃO GERAL

Introdução

Este checkpoint integra verificações automáticas de segurança ao fluxo de integração contínua. Cada push e pull request aciona uma análise de segredos, uma análise estática de JavaScript e uma análise de vulnerabilidades da imagem Docker. Detectar problemas cedo reduz o risco de publicar credenciais, executar código não confiável e distribuir componentes vulneráveis.

Arquitetura do pipeline

O arquivo **.github/workflows/security.yml** define três jobs independentes. O Gitleaks lê o histórico da branch após checkout completo; o Semgrep inspeciona o código-fonte com uma regra local; o Trivy constrói a imagem do Dockerfile e consulta bases de CVEs. Jobs independentes permitem ver qual barreira falhou sem confundir o tipo de problema.

push / pull request → gitleaks + semgrep + trivy

Política: Gitleaks bloqueia segredos; Semgrep bloqueia eval; Trivy exibe todas as severidades e bloqueia CRITICAL.

Aplicação analisada

O repositório começou vazio. Foi criada uma aplicação HTML/CSS/JavaScript sem dependências de produção, servida por Nginx sem privilégios na porta interna 8080. O painel apresenta as três barreiras e uma lista local de estudo salva no navegador. O progresso da interface não representa o estado dos jobs.

Estado de validação

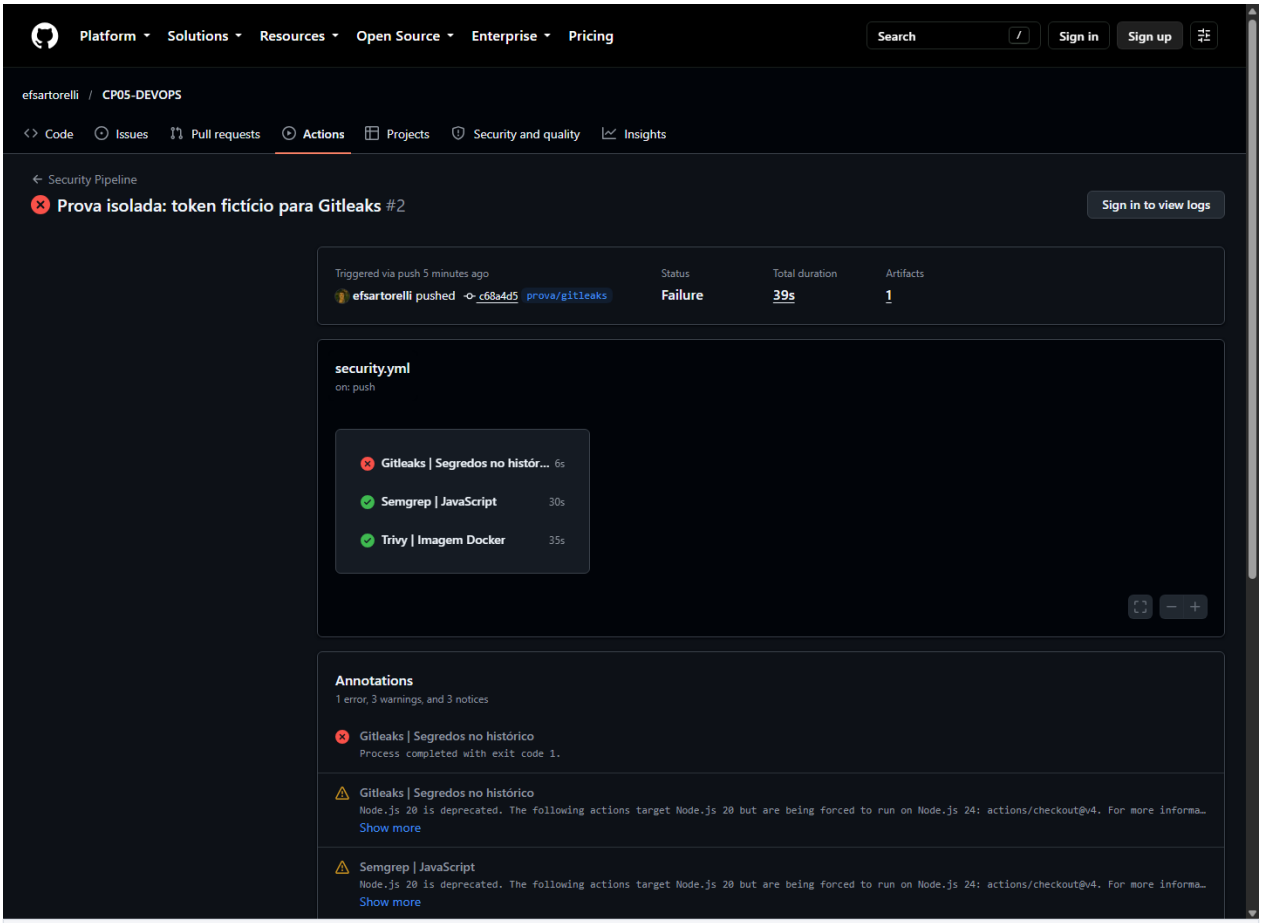
A aplicação respondeu por HTTP local, o JavaScript passou na verificação de sintaxe e o YAML passou no actionlint. No run 38462913681 da main, os três jobs passaram; a imagem foi construída e o contêiner respondeu HTTP 200. Os runs isolados falharam respectivamente no Gitleaks e no Semgrep. Um PR de teste também acionou os três jobs e foi fechado sem merge.

02 / PRIMEIRA BARREIRA

Gitleaks: segredos no histórico

O Gitleaks 8.30.0 procura credenciais em commits. O job usa **fetch-depth: 0**, estende as regras padrão e acrescenta uma assinatura para o token fictício da prova. A branch **prova/gitleaks** contém o commit isolado c68a4d5, sem merge na main.

No run 38461907382, o job Gitleaks falhou em 13/10/2026 às 20:19 UTC. O log autêntico mostra **RuleID fiap-cp05-fake-token**, **leaks found: 1** e **exit code 1**. O valor foi redigido. Isso confirma uma falha esperada de segurança. Segredos reais expostos exigiriam revogação ou rotação, mesmo após remover o arquivo em novo commit.



Trecho do log do GitHub Actions

RuleID: fiap-cp05-fake-token | leaks found: 1 | Process completed with exit code 1.

Captura autêntica do run no GitHub Actions. O detalhe da regra está em evidencias/01-gitleaks-github.txt. URL: github.com/efsartorelli/CP05-DEVOPS/actions/runs/38461907382

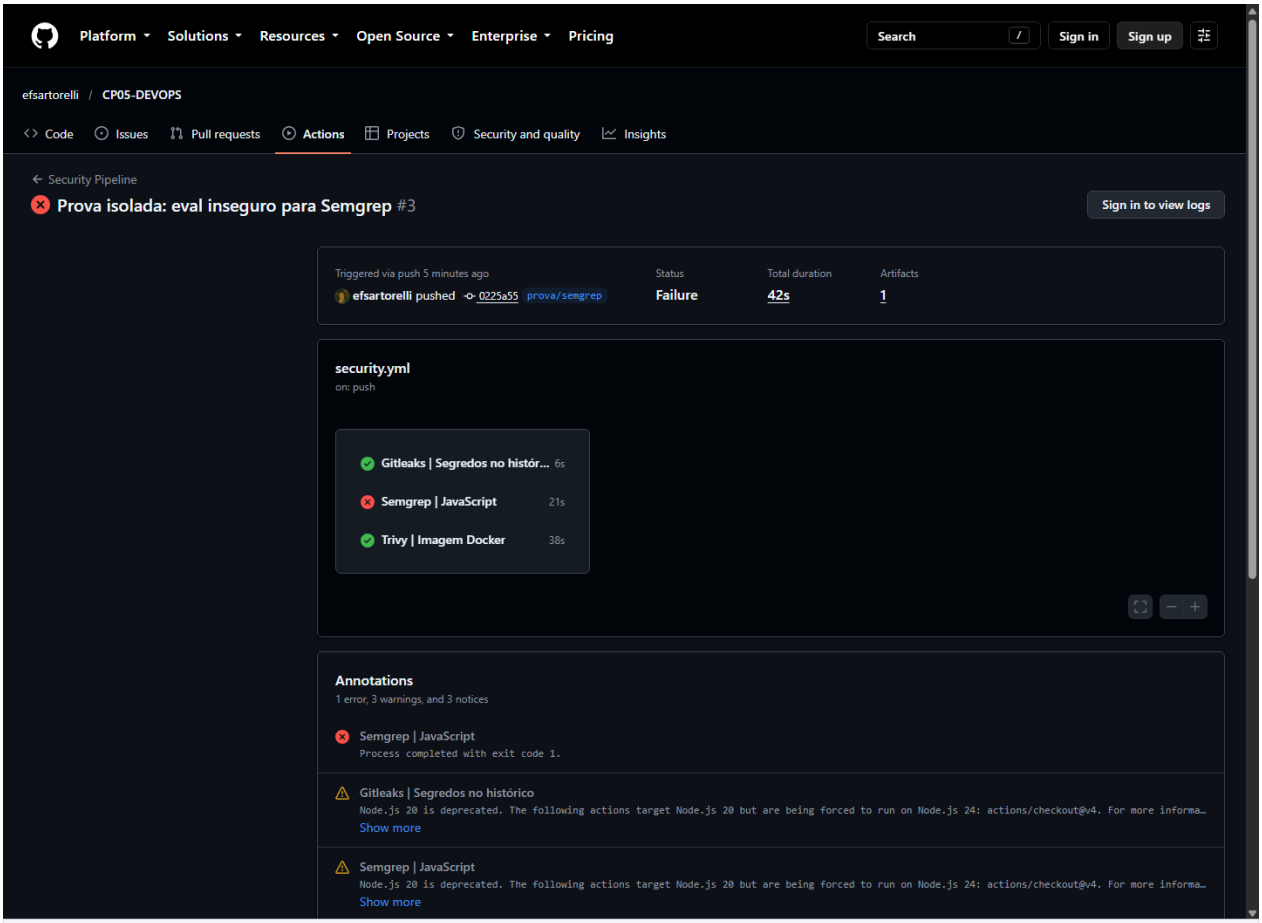
03 / SEGUNDA BARREIRA

Semgrep: análise estática

O Semgrep 1.140.0 analisa JavaScript. A regra local **fiap-cp05-no-eval**, criada para esta demonstração, identifica **eval(...)**; a opção **--error** torna o achado bloqueante. A branch **prova/semgrep** recebeu o exemplo abaixo, que não é executado:

```
function exemploInseguro(entrada) { return eval(entrada); }
```

No run 38461915387, o job Semgrep falhou em 13/10/2026 às 20:19 UTC com **1 finding (1 blocking)**, regra **semgrep.fiap-cp05-no-eval** em app.js e **exit code 1**. A main passou sem achados. eval interpreta texto como código JavaScript e pode executar entrada influenciada por terceiros.



Trecho do log do GitHub Actions

semgrep.fiap-cp05-no-eval | Findings: 1 (1 blocking) | Process completed with exit code 1.

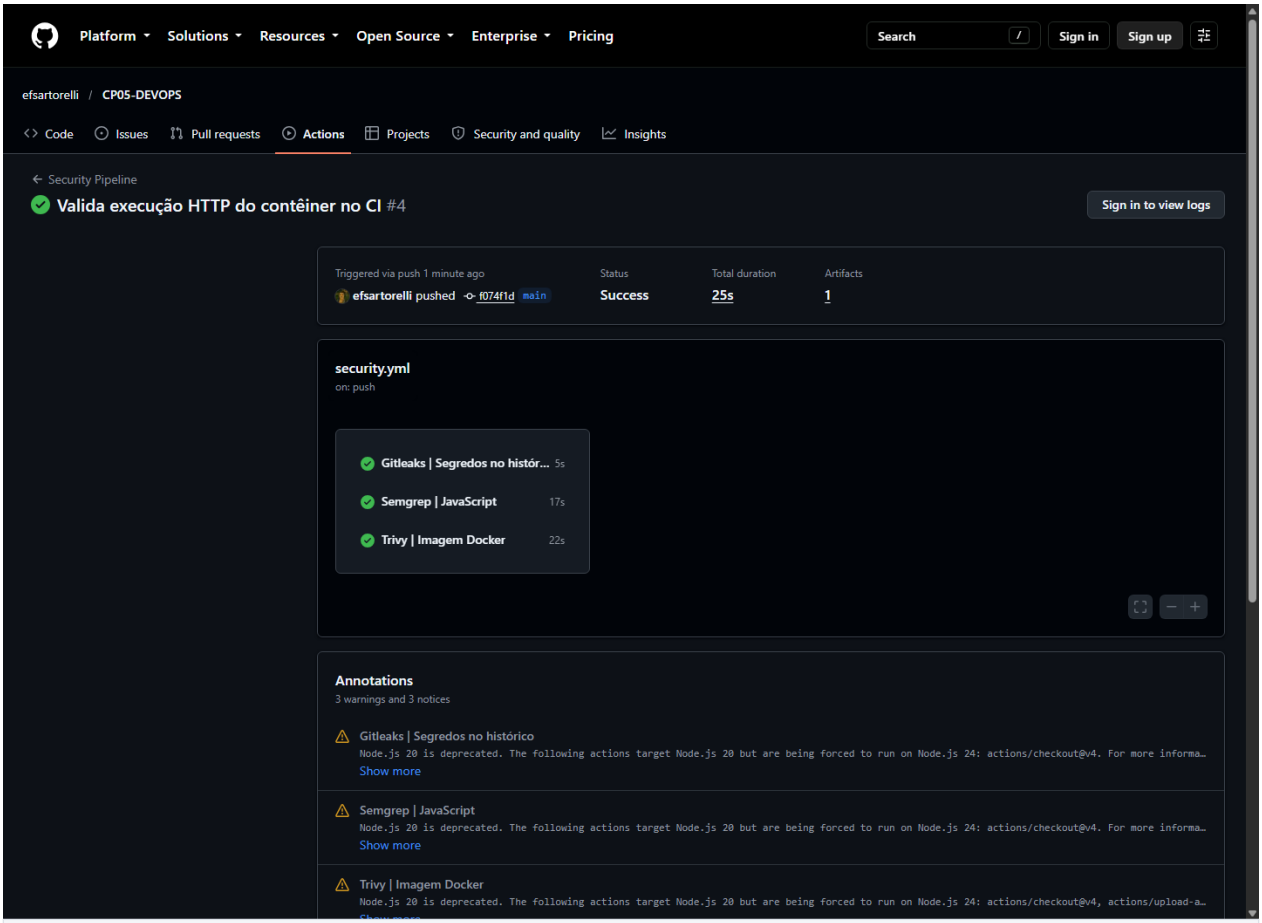
Captura autêntica do run no GitHub Actions. O achado detalhado está em evidencias/02-semgrep-github.txt. URL: github.com/efsartorelli/CP05-DEVOPS/actions/runs/38461915387

04 / TERCEIRA BARREIRA

Trivy: vulnerabilidades da imagem

O job construiu **fiap-cp05:f074f1da9053a1a97aa7682e18f725942dfa72fc** a partir do Dockerfile e confirmou resposta HTTP 200 na porta 8080 do contêiner. O Trivy examinou a imagem construída, publicou um JSON como artefato, exibiu a tabela nos logs e aplicou a política de bloqueio para **CRITICAL**.

No run 38462913681, a tabela apresentou **1 CVE MEDIUM** e **0 CRITICAL**: **CVE-2026-85091** no pacote zlib 1.3.2-r0, com correção indicada em 1.3.2-r1. O job passou porque MEDIUM está abaixo do limite definido. Isso não significa ausência de vulnerabilidades; o pacote deve ser reavaliado em atualização segura da imagem-base.



Trecho do log do GitHub Actions

Total: 1 (MEDIUM: 1, CRITICAL: 0) | zlib | CVE-2026-85091 | Aplicação respondeu HTTP 200 dentro do contêiner.

Captura autêntica do run na main. Tabela e build: evidencias/03-trivy-github.txt; relatório completo: evidencias/03-trivy-github-report.json. URL: github.com/efsartorelli/CP05-DEVOPS/actions/runs/38462913681

05 / ENCERRAMENTO

Conclusão

As três ferramentas protegem etapas diferentes: Gitleaks examina exposição de credenciais no histórico; Semgrep identifica um padrão de código inseguro; Trivy relaciona pacotes da imagem a CVEs. O pipeline executou os três jobs na main com sucesso e bloqueou as duas sabotagens em branches isoladas. O Trivy registrou uma CVE MEDIUM na imagem construída e manteve o job aprovado pela política que bloqueia CRITICAL.

Repositório e execuções

Repositório: https://github.com/efsartorelli/CP05-DEVOPS

Gitleaks (falha esperada): https://github.com/efsartorelli/CP05-DEVOPS/actions/runs/38461907382

Semgrep (falha esperada): https://github.com/efsartorelli/CP05-DEVOPS/actions/runs/38461915387

Trivy e main (sucesso): https://github.com/efsartorelli/CP05-DEVOPS/actions/runs/38462913681

pull_request (teste, PR #1 fechado): https://github.com/efsartorelli/CP05-DEVOPS/actions/runs/38464011096

Commits das provas: prova/gitleaks em c68a4d5; prova/semgrep em 0225a55. Ambas as branches permanecem isoladas da main.

Integridade das evidências

As três imagens são capturas diretas das páginas dos runs no GitHub Actions. A visualização pública pede login para mostrar logs; por isso os logs autênticos foram baixados pela API autenticada e preservados ao lado das imagens. Os arquivos de sabotagem nunca foram mesclados à main.

Fontes técnicas

Gitleaks: github.com/gitleaks/gitleaks
Semgrep CLI: semgrep.dev/docs/cli-reference
Trivy Action: github.com/aquasecurity/trivy-action
Nginx sem privilégios: github.com/nginx/docker-nginx-unprivileged